week3_notebook_student

October 27, 2026

1 HS4002 Week 3

1.1 Data Preparation

This week we need the `scipy` library for skewness and kurtosis statistics.

Make sure `gss2022_mini.csv` is in the same folder.

[ ]:
```
# !pip install pandas numpy scipy plotnine

import pandas as pd
import numpy as np
import plotnine as p9
import warnings
warnings.filterwarnings('ignore')
from scipy.stats import skew, kurtosis
```

1.2 Select variables of interest

[ ]:
```
raw_df = pd.read_csv("gss2022_mini.csv")

chosen_variables = [
        "id", "yrlvmus" ...]

df = raw_df[chosen_variables].copy()
```

1.3 Remove all rows with missing values

[ ]:
```
df = df.dropna()
```

1.4 Recode categorical variables into binary variables

`1 = attended / participated`, `0 = did not`. Refer to the GSS Codebook for coding.

[ ]:
```
df['binary_lvmus']  = np.where(df['yrlvmus']  == 1, 1, 0)
df['binary_artxbt'] =
df['binary_movie']  =
df['binary_creat']  =
```

1.5 Recode income into a continuous variable

`income16` is an ordinal variable (income brackets). Standard practice assigns each person the midpoint of their bracket. In Python, a dictionary lookup is much cleaner than nested `ifelse()` calls.

[ ]:
```
# Recode income16 (ordinal brackets) to continuous midpoint values
# This replaces the deeply nested ifelse() chain from the R version
income_map = {
        1: 500,    2: 2000,
        ...
}
df['income_cont'] = df['income16'].map(income_map)
```

1.6 Make a new variable for the omnivorousness index

Sum the four binary cultural participation variables.

[ ]:
```
df = df.assign(
        omni=(
                df['binary_lvmus']
                + df['binary_artxbt']
                + df['binary_movie']
                + df['binary_creat']
        )
)
```

1.7 Produce 3 samples

Sizes n = 30 (small), n = 100 (medium), and n = 500 (large).

[ ]:
```
np.random.seed(47)
df_small  = df.sample(n=30,  random_state=47)
df_medium = df.sample(n=100, random_state=47)
df_large  =
```

2 Compare Distributions of Categorical Variables Across Samples

2.1 What proportion attended live music last year?

[ ]:
```
df_small['binary_lvmus'].value_counts(normalize=True)
```

2.2 What proportion visited an art exhibit last year?

[ ]:
```

```

2.3 Plot a bar chart for movie-watching (large sample)

[ ]:
```
p = (
        p9.ggplot(df_large) +
        p9.geom_bar(p9.aes(x='factor(binary_movie)'), width=0.3) +
        p9.theme_light() +
        p9.labs(x='Movie Watching (1 = Yes)', y='Count',
                        title='Distribution of Movie Watching Over Past Year',
                        subtitle='Data from GSS 2022')
)
p.save('bar_movies.png', width=15, height=10.5, units='cm')
p
```

2.4 Plot a histogram of the omnivorousness index

[ ]:
```
p = (
        p9.ggplot(df_large) +
        p9.geom_histogram(p9.aes(x='omni'), binwidth=1) +
        p9.theme_light() +
        p9.labs(x='Omnivorousness Index', y='Count',
                        title='Distribution of Omnivorousness',
                        subtitle='Data from GSS 2022')
)
p.save('hist_omni.png', width=15, height=10.5, units='cm')
p
```

3 Compare Distributions of Two Continuous Variables

We compare household income with occupational prestige using the large sample.

3.1 Scale the variables

Standardize so both variables are on the same scale (mean = 0, SD = 1). Equivalent to R's `scale()`.

[ ]:
```
df_large = df_large.copy()
df_large['scaled_income'] = (df_large['income_cont'] - df_large['income_cont'].
 ↪mean()) / df_large['income_cont'].std()
df_large['scaled_prestg'] =
```

3.2 Plot the distributions of each variable

[ ]:
```
p = (
        p9.ggplot(df_large) +
        p9.geom_histogram(p9.aes(x='scaled_income'), bins=50, fill='orangered')␣
 ↪+
        p9.theme_light() +
        p9.labs(x='Normalized Values', y='Count',
```

```
                        title='Distribution of Household Income',␣
 ↪subtitle='Data from GSS 2022')
)
p.save('hist_income.png', width=15, height=10.5, units='cm')
p
```

[ ]:

3.3 Calculate Skewness of each variable

Skewness > 0 is right-skewed (long right tail); < 0 is left-skewed.

[ ]:
```
print('Skewness of Income (scaled):',  skew(df_large['scaled_income']))
```

3.4 Calculate Kurtosis of each variable

`scipy.stats.kurtosis` with `fisher=False` gives Pearson kurtosis (same as R's `moments::kurtosis`). A normal distribution has kurtosis 3 (mesokurtic); > 3 is leptokurtic.

[ ]:
```
# fisher=False gives Pearson kurtosis - equivalent to R's moments::kurtosis()
print('Kurtosis of Income (scaled):',  kurtosis(df_large['scaled_income'],␣
 ↪fisher=False))
```